

VAMOS: A Framework for Fast, Configurable and Accessible Multiobjective Optimization in Python

Anonymous submission

Abstract—Python is widely used for empirical research in multi-objective optimization due to its accessibility and strong scientific ecosystem. However, most Python frameworks store populations as per-solution objects and keep critical computations in the interpreter, which prevents vectorization and significantly limits performance. This design gap makes Python-based metaheuristics much slower than their counterparts in compiled languages. In this paper we introduce VAMOS (Vectorized Architecture for Multiobjective Optimization Studies), a vectorized Python framework for multi-objective evolutionary algorithms (MOEAs). VAMOS maintains all population data in dense numerical arrays and dispatches hot-path operations to interchangeable numerical backends, decoupling algorithm logic from the underlying compute substrate. The framework implements eight MOEAs spanning dominance-based, decomposition-based, and indicator-based paradigms, and exposes rich component-level configurability, including interchangeable operators, external archives, and fine-grained parameter settings. VAMOS also integrates an automated algorithm-configuration module and provides a browser-based graphical interface to simplify use by non-experts. The results show that VAMOS achieves statistically equivalent levels of solution quality than those elicited by MOEAs implemented in state-of-the-art libraries, while reducing wall-clock time by large margins across all tested algorithms. These gains confirm that vectorized population models and backend-agnostic kernels at the core of VAMOS can deliver high performance in Python without compromising algorithmic behavior.

Index Terms—Evolutionary algorithms, multiobjective optimization, software framework, vectorization

I. INTRODUCTION

MULTI-OBJECTIVE optimization problems (MOPs) arise whenever trade-offs exist between conflicting objectives, requiring the computation of an accurate approximation of the Pareto-optimal set rather than a single optimum [1]. Such problems are common in engineering design, machine learning, scheduling, and resource allocation, among other domains. In this field, multi-objective evolutionary algorithms (MOEAs) remain a dominant solving approach due to their robustness, population-based behavior, and ability to return a set of non-dominated solutions in a single run [2].

The widespread adoption of MOEAs has been fueled in large part by the availability of software frameworks that lower the implementation barrier and facilitate reproducible experimentation. Early efforts such as *PISA* [3] established the concept of modular, platform-independent interfaces for multi-objective search. The *jMetal* framework [4] later consolidated a comprehensive Java-based framework that became a reference for algorithm development and research studies. More recently, *PlatEMO* [5] has provided a MATLAB platform with a large catalog of algorithms and test problems. In parallel,

the Python ecosystem has seen the emergence of libraries implementing and evaluating MOEAs for MOPs, such as *pymoo* [6], *jMetalPy* [7], *Platypus* [8], and *DEAP* [9], which leverage Python’s accessibility and rich scientific stack.

As Python has become a widely adopted platform for MOEA research and experimentation, three practical limitations hinder its broader adoption. First, the interpreted nature of Python, combined with libraries that keep hot-path computations inside the interpreter [7]–[9], makes the inner loops of pure-Python implementations inherently slower than those of frameworks written in compiled languages such as C++ or Java. Second, existing MOEA frameworks often provide limited configurability at the component level and do not allow attaching external archives, useful to allow the Non-dominated Sorting Genetic Algorithm II (NSGA-II) to handle problems with more than two objectives [10], with the exception of *jMetal* [4]. Third, most existing frameworks present a high entry barrier for non-expert users while simultaneously lacking the necessary flexibility for advanced researchers to achieve fine-grained algorithmic configuration.

To overcome these limitations, we introduce VAMOS, a Python framework for addressing MOPs with MOEAs designed around a simple principle: maintain the entire population state in dense numerical arrays and offload hot-path computations to vectorized or compiled backends. This architecture eliminates per-solution Python overhead and allows switching between NumPy, Numba, and C-based kernels through a single configuration parameter. Beyond performance, VAMOS provides broad component-level configurability, including interchangeable variation operators, flexible initialization schemes, steady-state and generational modes, and optional bounded or unbounded external archives. The framework implements eight MOEAs covering dominance-based, decomposition-based, and indicator-based paradigms, all built on a shared set of operators to ensure semantic consistency across algorithms. VAMOS also integrates automated algorithm-configuration tools and offers browser-based interfaces for problem definition, experiment execution, and results exploration. These design choices enable both early-stage and expert users to prototype, tune, and benchmark MOEAs efficiently while preserving reproducibility and cross-framework comparability. Specifically, the main contributions of VAMOS can be summarized as follows:

- a) *Performance*: VAMOS introduces a unified, array-centric computational model in which all decision and objective data are maintained in dense numerical structures. This design choice eliminates the per-individual overhead inherent to object-oriented representations and enables all

core operations (i.e., selection, variation, ranking, and indicator evaluation) to be executed by pluggable numerical backends.

- b) *Configurability*: Eight MOEA implementations with a high component-level configurability and an automated algorithm configuration module. Each algorithm is built from a shared set of modular components, providing consistent and controlled comparisons. Through a unified builder interface, users may specify initialization procedures, selection operators, crossover and mutation schemes, replacement policies, and archive configurations. VAMOS integrates an automated configuration layer that interfaces with established model-based and multi-fidelity tuners, enabling systematic exploration of high-dimensional parameter spaces without additional user effort.
- c) *Accessibility*: VAMOS is designed to support a broad spectrum of users, from non-specialists to experienced algorithm designers. The Python application programming interface (API) emphasizes clarity and minimal code while still exposing detailed configuration options for advanced experimentation. VAMOS also provides guided command-line utilities and VAMOS Studio, a browser-based graphical environment that provides assisted problem definition, live algorithm previews, and integrated facilities for inspecting Pareto fronts and analyzing experimental outcomes.

The paper is organized as follows. Section II reviews existing Python MOEA frameworks. Section III presents the VAMOS framework. Sections V, VI, and VII examine the main functionalities of VAMOS in terms of performance, configurability, and accessibility. Finally, Section VIII draws conclusions and outlines directions for future work.

II. RELATED WORK

In this section, we review the most relevant Python libraries for MOEAs, namely *pymoo*, *jMetalPy*, *DEAP*, *Platypus*, and *PyGMO*. Table I summarizes the main differences between the compared frameworks in terms of architectural features, algorithmic capabilities, configurability, and user-facing tooling. Among the compared frameworks, VAMOS is the only one combining a dense array population model, pluggable computation backends, and a graphical user interface. *PyGMO* is included for capability context, but in the performance section we treat it separately as a compiled C++ baseline, rather than as a directly comparable pure-Python framework.

The current landscape of Python-based libraries for MOEAs reflects a balance between usability, extensibility, and computational efficiency. Among these libraries, *pymoo* [6] has become one of the most widely used due to its comprehensive catalog of algorithms and utilities. Its design emphasizes flexibility through subclassing and object-oriented abstractions, which facilitates customization but tightly couples algorithmic components to internal data structures. As a consequence, systematic vectorization of inner-loop operations is difficult, and performance remains bounded by Python-level overhead.

jMetalPy [7] follows a different rationale. As a Python adaptation of the Java-based *jMetal* framework [4], it inherits

TABLE I
COMPARISON OF PYTHON MOP FRAMEWORKS. ✓ FULL SUPPORT
○ PARTIAL/LIMITED SUPPORT ✗ ABSENCE

Feature	<i>pymoo</i> [6]	<i>jMetalPy</i> [7]	<i>DEAP</i> [9]	<i>Platypus</i> [8]	<i>PyGMO</i> [11]	VAMOS (proposed)
Performance indicators	✓	✓	○	✓	○	✓
Dense array model	○	✗	✗	✗	✗	✓
Pluggable backends	✗	✗	✗	✗	✗	✓
Parallelism	✓	✓	✓	✓	✓	○
Dominance-based	✓	✓	○	✓	✓	✓
Decomposition-based	✓	✓	✗	✓	✓	✓
Indicator-based	○	✓	✗	✓	✗	✓
Interchangeable operators	✓	✓	✓	○	✗	✓
Conf. steady-state mode	✓	✓	○	✗	✗	✓
External archive	✗	○	○	✓	✗	✓
Graphical user interface	✗	✗	✗	✗	✗	✓

a clear, component-based design and solid runtime monitoring capabilities. This structure is well suited for experimentation and algorithmic prototyping. However, like *pymoo*, it manages populations as collections of solution objects, prioritizing architectural clarity over numerical efficiency. This design choice ultimately limits its ability to offload heavy computations to optimized array backends.

DEAP [9] takes a more general-purpose approach. Instead of targeting MOEAs specifically, it offers a functional, modular toolkit for evolutionary algorithm development in the broadest sense. This grants considerable expressive power but leaves the construction of sophisticated multi-objective pipelines largely to the user. Its per-individual object representation further restricts opportunities for vectorization, placing it in a similar performance regime to *pymoo* and *jMetalPy* when used for multi-objective tasks.

Platypus [8] sits at the opposite end of the complexity spectrum. It is intentionally lightweight and accessible, making it attractive for beginners or for rapid experimentation. However, its simplicity comes at the cost of limited maintenance in recent years and a continued reliance on object-oriented population models, which again hinders performance scalability.

Finally, rather than providing a pure Python implementation, *PyGMO* [11] wraps the C++ *pagmo2* library, delivering substantially faster runtimes through compiled kernels and parallel execution features. While powerful, its architecture is tailored primarily to continuous single-objective optimization, and its multi-objective capabilities offer only limited configurability. Moreover, algorithmic components such as archiving strategies are tightly integrated, reducing its suitability as a research testbed for flexible MOEA design. For these reasons, *PyGMO* serves as a compiled baseline for runtime comparison than as a directly comparable Python framework.

Distinct from these central processing unit (CPU)-based Python libraries, recent specialized solutions target divergent architectures and problem scopes. For example, *EvoX* [12] provides a functional programming model specifically tailored for mapping evolutionary workloads onto distributed graphics processing unit (GPU) resources. Other contributions focus on specialized settings, such as coevolutionary constrained optimization [13], bi-knowledge transfer for multimodal problems [14], mixed-integer linear pipelines [15], and large language model (LLM)-assisted code generation [16].

Overall, while existing libraries span a wide spectrum between usability and performance, none of them explicitly decouples algorithmic logic from the numerical substrate to enable transparent CPU backend substitution. Furthermore, they either target specific niche scenarios or fail to combine a low entry barrier for domain experts with fine-grained configurability for researchers. This gap motivates VAMOS, which provides a vectorized, backend-agnostic architecture aimed specifically at MOEA studies and reproducible cross-framework benchmarking on standard hardware.

III. VAMOS FRAMEWORK

The design of VAMOS is motivated by three practical gaps in existing Python MOEA libraries:

a) *Performance*: As the population state remains in dense arrays throughout the optimization loop, hot-path computations are dispatched to vectorized or compiled kernels. As we will see in Section V, this design yields substantial runtime reductions across multiple MOEA variants and benchmark families without degrading solution quality. Moreover, switching the compute backend only requires a single-parameter change transparent to algorithm code. No other Python framework allows swapping the entire computational substrate, from array operations to compiled kernels, without modifying the algorithm implementation.

b) *Configurability*: Most frameworks provide a small set of high-level parameters (population size, crossover/mutation probabilities), but do not expose fine-grained control over offspring batch size, archive attachment, or different initialization methods. VAMOS exposes all of these as first-class configuration options through a builder API. In addition, it provides a unified tuning interface supporting random search and model-based backends such as Optuna [17], Sequential Model-based Algorithm Configuration (SMAC3) [18], and Bayesian Optimization and HyperBand (BOHB) [19].

c) *Accessibility*: The majority of other frameworks require importing separate modules for algorithm selection, problem definition, operators, and termination, then wiring them together. This procedure assumes familiarity with MOEA terminology and framework implementation details, which poses a barrier for domain experts who need to solve MOPs but lack a background in MOEAs. As later shown in Section VII, VAMOS addresses this gap at multiple levels: (i) two lines of code are sufficient to run a complete search with sensible default parameters; (ii) a problem factory turns any Python callable into a problem object without subclassing; (iii) guided command-line tools and VAMOS Studio provide a clear on-boarding path for non-experts; and (iv) when a finer control is needed, the builder API ensures that VAMOS scales from novice to expert use without a different API.

IV. VAMOS ARCHITECTURE

The core architecture of VAMOS is organized into the four layers shown in Figure 1: (i) the *Foundation Layer*, which provides compute kernels, problem definitions, quality indicators, encodings, constraints, and evaluation backends; (ii) the *Engine Layer*, implementing MOEAs on top of shared

components, variation operators, external archives, and a tuning subsystem; (iii) the *Experiment Layer*, which exposes the unified `optimize()` entry point, command-line interface (CLI) tools, benchmarking utilities, and results analysis; and (iv) the *User Experience (UX) Layer*, providing VAMOS Studio (an interactive dashboard), a visual problem builder, a Results Explorer, and an on-boarding wizard.

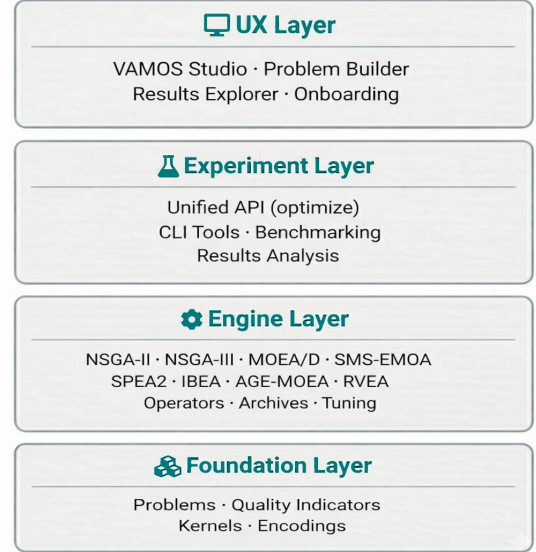


Fig. 1. VAMOS four-layer architecture.

A. Foundation Layer

Given a MOP with n decision variables and m objectives, the core design choice in VAMOS is to represent a population of size N as a pair of dense arrays, one for decision variables $X \in \mathbb{R}^{N \times n}$ and another for objective values $F \in \mathbb{R}^{N \times m}$. In this way, the algorithmic logic is separated from numerical kernels that implement the corresponding core computations. Specifically, VAMOS provides three interchangeable backends to avoid operating over per-individual Python objects. Each backend can be selected through a single configuration parameter, transparently to algorithm code:

- 1) *NumPy* as a baseline backend, enabling vectorized array operations [20].
- 2) *Numba* as a just-in-time (JIT) compilation of hot operators and utilities, reducing Python overhead [21].
- 3) *MooCore* as an optional C-backend for Pareto ranking and hypervolume-based utilities [22].

In addition to accelerating arithmetic kernels, VAMOS reduces overhead by minimizing Python-side allocations, which allows variation operators to reuse pre-allocated workspaces and algorithms to maintain state as contiguous arrays. This design is particularly beneficial in inner-loop routines such as tournament selection, non-dominated sorting, crowding-distance computation, and archive updates.

Fig. 2 provides an intuitive view of how these frameworks represent and manipulate a population. All frameworks implement the same selection/variation/survival loop, but differ in internal data model and hot-loop granularity. In VAMOS (left) the population is stored in dense arrays (X , F) and the

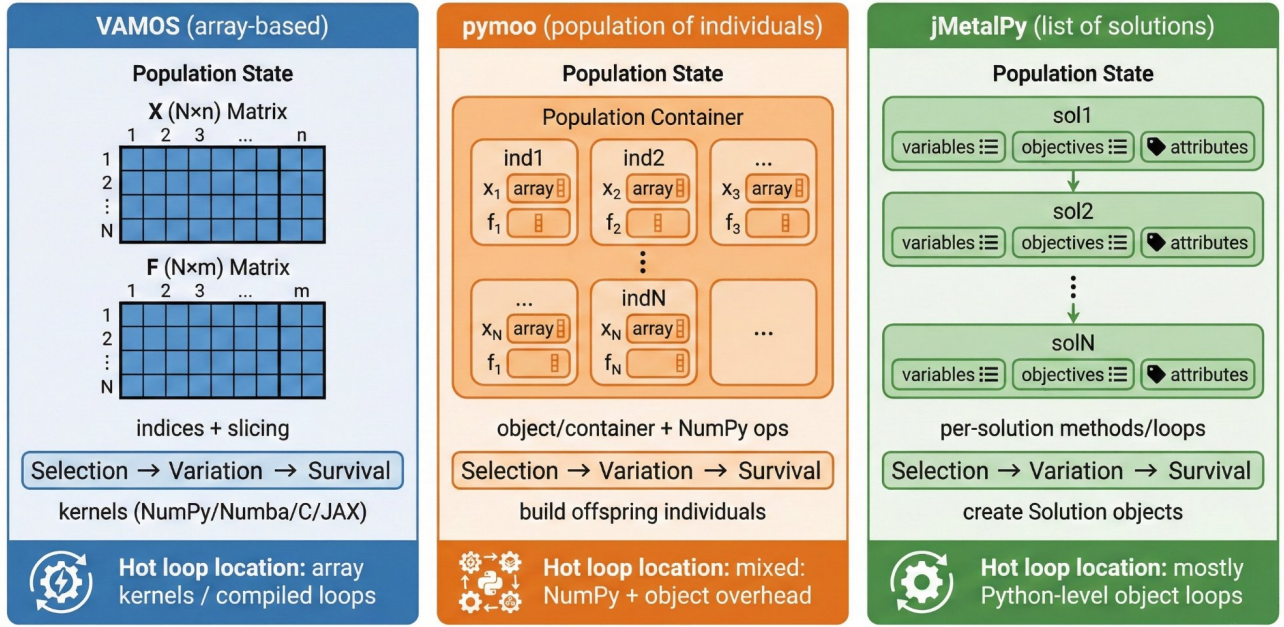


Fig. 2. Comparing population representation across Python MOEA frameworks.

selection/variation/survival operations are applied via array-oriented kernels (*NumPy/Numba/MooCore*). In contrast, *pymoo* (center) and *jMetalPy* (right) manage the population as a collection of solution objects. Specifically, *pymoo* uses a population container of individual objects backed by NumPy arrays, mixing array computation with object overhead, and *jMetalPy* manages a list of solution objects with per-solution method calls.

Beyond kernels, the Foundation Layer hosts the problem abstraction, a base `Problem` class that declares the number of decision variables, objectives, and optional constraints. A problem registry enables lookup by name (e.g., “zdt1”, “dttlz2”), and a `make_problem` factory converts any Python callable into a problem object without subclassing. Built-in benchmark families include ZDT [23], DTLZ [24], WFG [25], ZCAT [26], the Congress on Evolutionary Computation (CEC) 2009 test suite [27], and the real-world problem collections RE (Real-World) [28] and RWA (Real-World Applications) [29]. The layer also provides five variable encodings (real, binary, integer, permutation, and mixed), a constraint-handling module with a declarative domain-specific language (DSL), and quality indicators (hypervolume, generational distance, inverted generational distance, and spread).

B. Engine Layer

VAMOS implements the eight MOEAs shown in Table II: Non-dominated Sorting Genetic Algorithm II (NSGA-II), Non-dominated Sorting Genetic Algorithm III (NSGA-III), Strength Pareto Evolutionary Algorithm 2 (SPEA2), Adaptive Geometry Estimation-based MOEA (AGE-MOEA), Multi-objective Evolutionary Algorithm Based on Decomposition (MOEA/D), Reference Vector Guided Evolutionary Algorithm (RVEA), S-metric Selection Evolutionary Multiobjective Optimization Algorithm (SMS-EMOA), and Indicator-Based Evo-

lutionary Algorithm (IBEA). These algorithms cover the three major paradigms: dominance, decomposition, and indicator-based. All eight algorithms are built on top of the shared selection/variation/survival components, ensuring consistent operator semantics and enabling component-level substitution without algorithm-specific code changes. Consequently, users of VAMOS can compare different solving paradigms under identical infrastructure and configuration conventions. In addition to the selection of the algorithm, Table II summarizes the number of available crossover (#Cx) and mutation (#Mx) operators, algorithm-specific parameters, and the total number of configurable parameters (base + conditional) in the tuning configuration space (#Params). The reported operator counts aggregate all supported encodings, whereas the parameter counts aggregate the unique parameter names exposed by the encoding-specific builders. In any single configuration with a fixed encoding, only a subset of these parameters is simultaneously active because many are conditional on the selected operators. The parameter counts reflect the tuning configuration spaces used by the built-in auto-tuner (up to 36 unique parameters for NSGA-II across encodings), illustrating the breadth of component-level configurability that VAMOS exposes as first-class configuration options for systematic tuning. All algorithms in VAMOS provide a uniform `run(problem, termination, seed, eval_backend, live_viz)` interface, to which the high-level `optimize()` function delegates after resolving defaults and constructing the chosen operators. Furthermore, all algorithms support an *ask-tell* loop for streaming and interactive execution modes.

VAMOS supports real-valued, binary, integer, permutation, and mixed-variable encodings through a variation pipeline that composes selection, crossover, mutation, and optional repair operators. For continuous domains, the default configuration uses Simulated Binary Crossover (SBX) and Polynomial Mu-

TABLE II
OVERVIEW OF THE EIGHT MOEAS IMPLEMENTED IN VAMOS AND THEIR CONFIGURABLE PARAMETER SPACES.

Algorithm	Ref.	Paradigm	#Cx	#Mx	Specific Parameters	#Params
NSGA-II	[30]	Dominance	25	24	Offspring ratio, repair, external archive (mode, prune policy)	36
NSGA-III	[31]	Dominance	21	22	Reference directions (auto-computed)	26
SPEA2	[32]	Dominance	21	22	Archive size, k -nearest neighbors	28
AGE-MOEA	[33]	Dominance	21	22	Archive policies (crowding, hypervolume, ε -grid, hybrid)	25
MOEA/D	[34]	Decomposition	17	20	Aggregation (Tchebycheff, weighted sum (WS), penalty-based boundary intersection (PBI)), T , δ , η	23
RVEA	[35]	Decomposition	21	22	Partitions, α decay, adaptation frequency	27
SMS-EMOA	[36]	Indicator	21	22	Reference point (adaptive)	26
IBEA	[37]	Indicator	21	22	Indicator type, scaling factor κ	28

tation (PM), with mutation probability typically set to $1/n$ where n is the number of decision variables. Binary and integer encodings use standard crossover (one-point, two-point, or uniform) and bit-flip or bounded mutation, while permutation encodings provide order and cycle crossover with insertion mutation. Mixed-variable problems combine encoding-specific operators transparently. Encodings are normalized internally to ensure consistent operator resolution and avoid silent configuration mismatches.

The engine layer also provides optional external archives, both bounded (with crowding-distance or hypervolume-based pruning) and unbounded, that accumulate non-dominated solutions independently of the evolving population. This decouples the population-based search from the elite set, which is particularly useful for problems with more than two objectives [10]. Common quality indicators are provided by the foundation layer, including hypervolume, generational distance (GD), inverted generational distance (IGD), and spread. Hypervolume computation can leverage the MooCore C backend when available¹, with pure-Python fallback implementations for low-dimensional MOPs.

C. Experiment Layer

The experiment layer hosts the unified `optimize()` entry point that resolves defaults, instantiates the algorithm and problem, and delegates the search to the engine layer. It also provides a command-line interface with subcommands for running optimizations, hyperparameter tuning, ablation studies, diagnostics, and for launching the VAMOS Studio. A benchmark runner executes standardized multi-seed studies under a shared protocol (fixed problem definitions, evaluation budgets, and seeds) and emits machine-readable artifacts (per-seed objective values, summary CSVs, and LaTeX-ready tables), reducing transcription errors and enabling end-to-end reproducibility of experiments.

D. User Experience Layer

The user experience layer provides VAMOS Studio, a Panel-based [38] dashboard designed to make multi-objective optimization accessible to users with limited programming experience. It offers three main views: (i) a *Welcome* tab with a first-launch on-boarding wizard and quick-reference tables for the CLI and Python API; (ii) a *Problem Builder* that lets users

write objective and constraint functions in the browser, adjust bounds and algorithm settings, and observe the Pareto front update live after each preview run; and (iii) an *Explore Results* tab for loading completed studies, comparing MOEAs, and ranking solutions with multi-criteria decision-making methods. The *Problem Builder* ships with domain-specific templates (engineering design, machine learning, and scheduling) so that domain experts can start from a realistic example rather than a blank editor. Generated problems can be exported as standalone Python scripts or run directly through the API. In addition to Studio, the UX layer provides user-facing analysis and visualization utilities for inspecting optimization results, including statistical comparisons of algorithm runs (e.g., Wilcoxon tests and effect sizes) and helpers for plotting Pareto fronts and convergence traces.

V. PERFORMANCE

This section describes the experimental setup used to evaluate VAMOS, which is designed to assess its performance when compared with other Python frameworks for MOPs with MOEAs. To maintain a focused and controlled analysis, we restrict our study to continuous optimization problems. The remainder of this section outlines the benchmark suite, explains the rationale behind the selection of comparison frameworks and algorithms, details the semantic-alignment protocol used to ensure cross-framework fairness, and describes the procedures for runtime measurement and solution-quality assessment.

A. Compared Frameworks and Algorithms

Although VAMOS implements eight MOEAs, our cross-framework comparison focuses on those with the closest semantic matches across libraries: NSGA-II, SMS-EMOA, and MOEA/D. This scope avoids confounding effects from missing implementations or materially different operator and termination semantics in specific frameworks. Furthermore, these algorithms are representative of the three MOEA groups distinguished by Zhang *et al.* [39]: dominance-based (NSGA-II), performance-indicator-based (SMS-EMOA), and decomposition-based (MOEA/D).

In the case of NSGA-II, we compare three variants: the baseline (i.e., standard generational NSGA-II), a steady-state version, and an unbounded archive-based version. NSGA-II is a generational genetic algorithm, but some studies [40] have shown that a steady-state version (i.e., the offspring population contains only one solution) produces fronts with improved

¹MooCore is an optional dependency because it requires C compilation, which may not be available on all platforms.

diversity compared to the standard version. NSGA-II has traditionally been considered a poorly performing algorithm for problems with more than two objectives, but it was reported in [10] that incorporating an unbounded external archive improves the performance of NSGA-II on three-objective problems. Both variants enhance the base NSGA-II but at the cost of increased computational overhead.

In this comparison, we include *DEAP*, *Platypus*, and *PyGMO* only in an experiment that involves the standard generational NSGA-II. For the remaining experiments, our choice of algorithms restricts the comparison of VAMOS to the two previously analyzed pure-Python frameworks with the closest semantic matches, namely *pymoo* and *jMetalPy*. *DEAP* does not include built-in implementations of SMS-EMOA or MOEA/D, *Platypus* lacks SMS-EMOA and has seen reduced maintenance activity in recent years, and *PyGMO* does not include MOEA/D and is restrictive in terms of algorithmic configurability.

B. Benchmark Suites

We evaluate runtime and solution quality on widely used synthetic benchmarks: ZDT1–4 and ZDT6 [23] (two objectives; ZDT5 is excluded because it uses a binary encoding), DTLZ1–7 [24] (three objectives), WFG1–9 [25] (two objectives) and the recent ZCAT test suite [26] (two objectives). Unless otherwise stated, each (*problem*, *framework*) configuration is executed for 50,000 objective evaluations with 30 independent seeds. We have fixed the number of evaluations to simplify the comparisons, although it is well-known that the stopping condition to solve the ZDT and bi-objective WFG benchmarks is typically 20,000–25,000 evaluations.

C. Cross-Framework Semantic Alignment

Fig. 3 summarizes the benchmark protocol and semantic-alignment workflow used to ensure cross-framework fairness and to support the “same quality, faster” claim for the aligned NSGA-II comparison.

To ensure comparability across frameworks, we standardize the algorithmic settings of NSGA-II: population size $N = 100$, SBX crossover probability $p_c = 1.0$ and distribution index $\eta_c = 20$, polynomial mutation distribution index $\eta_m = 20$, mutation probability $p_m = 1/n$, and binary tournament selection. For each framework we map these settings to the closest available implementation. The initial population is filled with randomly initialized solutions.

For SMS-EMOA, we use the same common settings as NSGA-II. Compared with this algorithm, its main differences are the use of hypervolume contribution as density estimator, random parent selection, and a steady-state ($\mu + 1$) loop (one offspring per iteration). However, the hypervolume-contribution reference point is not defined consistently across frameworks: VAMOS and *jMetalPy* compute contributions in raw objective space with an adaptive reference point $r_j = \max_i F_{ij} + 1$ for each objective j , where F contains the objective values of the current population, while *pymoo* normalizes objectives and uses a fixed shifted reference point

$r = \mathbf{1} + \epsilon$ (we set $\epsilon = 1$). We therefore report the cross-framework SMS-EMOA comparison acknowledging that the reference-point definitions differ across libraries.

For MOEA/D, we set population size $N = 100$, weight vectors, neighborhood size $T = 20$, neighborhood selection probability $\delta = 0.9$, and replacement limit $\eta = 20$. We use penalty-based boundary intersection (PBI) scalarization with $\theta = 5$ and the SBX and polynomial mutation operators with the same parameters as NSGA-II. The weight vectors for the three-objective problems are generated using the Riesz s -energy method proposed in [41], available at <https://www.egr.msu.edu/coinlab/blankjul/uniform/>.

This workflow follows standard practice in comparative MOEA studies (independent runs, quality indicator computation, and structured reporting) and is consistent with the experimentation methodology described in *jMetalPy* [7]. Our main extension is to make the *inputs* to this workflow comparable across frameworks by enforcing semantic alignment of both problem definitions and operator probabilities.

It is important to note that our experimental protocol aligns *semantics* (not only parameter names) and enforces consistent benchmark definitions. For example, the polynomial mutation operator in *pymoo* exposes both an individual-level probability (`prob`) and a per-variable probability (`prob_var`); to match the standard “ $p_m = 1/n$ per decision variable” setting used by *jMetalPy*, we set `prob = 1` and `prob_var = 1/n`.

For the WFG benchmark we adopt the canonical parameterization of *pymoo* for two objectives ($k = 4$, $l = 20$ for $n = 24$) and pass parameters explicitly when a toolkit exposes different defaults. While full equivalence cannot be guaranteed due to framework-specific details (tie-breaking, boundary handling, duplicate elimination, etc.), these choices aim to make the comparison as fair as possible by removing confounders unrelated to algorithmic overhead or numerical kernel differences.

D. Runtime Measurement

All runtimes are measured wall-clock using high-resolution timers and include algorithm overhead and objective evaluations. For *Numba*-accelerated backends, we distinguish two timing policies: (i) *Warm* timings exclude one-time JIT compilation by performing a short warmup run in the same process before starting the timer (2,000 warmup evaluations; not counted in the reported time); and (ii) *cold* timings measure the first run from a fresh process and therefore include JIT compilation overhead. Unless otherwise stated, we report warm timings to reflect steady-state throughput; cold-start costs can be reproduced with the provided scripts. Experiments were run on a computer with an Intel Core Ultra 9 185H CPU (16 cores, 22 logical processors; up to 5.1 GHz) and 32 GB random-access memory (RAM). We used Python 3.12.3 and evaluated VAMOS 0.1.0 (*NumPy* 2.3.5 with *OpenBLAS* 0.3.30; *Numba* 0.63.1) against *pymoo* 0.6.1.6 and *jMetalPy* 1.9.0.

E. Quality Indicators

Hypervolume (HV), denoted $HV(A, r)$, measures, for minimization problems, the Lebesgue measure of the region dominated by an approximation set A and bounded by a reference

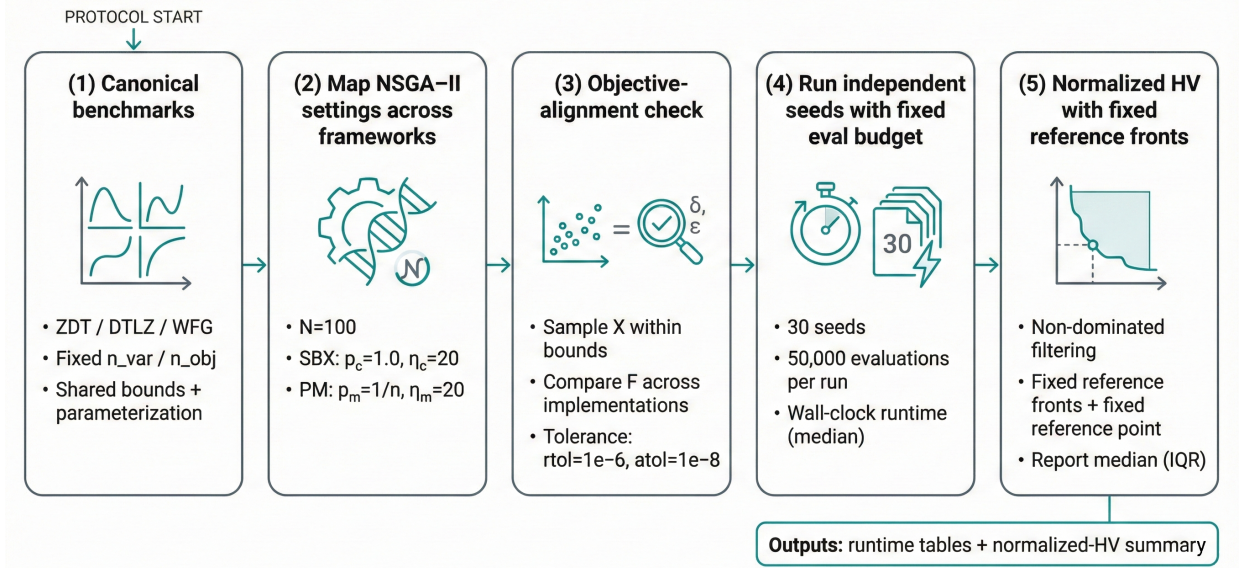


Fig. 3. Benchmark protocol and semantic alignment: map equivalent NSGA-II settings across frameworks, run independent seeds under a fixed evaluation budget, and compute normalized hypervolume using fixed reference fronts.

point r [42]. Since HV is scale-dependent and sensitive to the choice of r , naive implementations can produce values that are not comparable across runs (e.g., if r is expanded separately for each run) and can be misleading when dominated solutions are included. To ensure a stable and comparable quality metric, we adopt the following protocol:

- 1) *Non-dominated filtering*: for every framework, we compute HV on the final non-dominated set only.
- 2) *Fixed reference Pareto fronts*: for each benchmark problem, we store a dense reference Pareto front P_{ref} . For ZDT and DTLZ (except DTLZ7), these fronts are generated analytically; for DTLZ7 and WFG, we generate P_{ref} by dense sampling followed by non-dominated filtering.
- 3) *Fixed reference point*: for a minimization problem with m objectives, we define the reference point component-wise as

$$r_j = \max_{y \in P_{\text{ref}}} y_j + \epsilon, \quad j = 1, \dots, m,$$

where y_j denotes the value of objective j for a point y on the reference front. That is, for each objective, r_j is set slightly worse than the worst value attained on P_{ref} . We disallow any run-dependent expansion of r .

- 4) *Normalization*: we report $\text{HV}(A, r) / \text{HV}(P_{\text{ref}}, r)$, yielding a dimensionless score typically in $[0, 1]$ and reducing sensitivity to objective scaling.

For WFG problems, the reference front is necessarily an approximation; we use large Pareto-set samples and non-dominated filtering, and we increase density for particularly sensitive cases (e.g., WFG2) to avoid underestimating $\text{HV}(P_{\text{ref}}, r)$ and obtaining normalized HV slightly above 1.

F. VAMOS Backend Comparison

In the first experiment, we compare the performance of NSGA-II using the different backends of VAMOS on the ZDT, DTLZ, WFG, and ZCAT families. For each family, we sum

the execution times of solving all the problems in the family and report the median runtime of repeating this process 30 times. Table III reports the obtained family-level medians.

TABLE III
VAMOS BACKEND COMPARISON: MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY

Backend	ZDT	DTLZ	WFG	ZCAT
<i>Numba</i>	4.89	8.14	14.33	87.43
<i>MooCore</i>	21.01	36.45	53.96	127.46
<i>NumPy</i>	35.14	68.86	52.68	177.06

Numba is the fastest backend across all four families. *MooCore* is $1.5\text{--}4.5\times$ slower than *Numba*, while the *NumPy* backend is $2.0\text{--}8.5\times$ slower, confirming that per-generation interpreted Python overhead dominates the cost in the absence of JIT compilation. The largest gap appears on DTLZ, where *NumPy* is $8.5\times$ slower than *Numba*; the smallest relative difference is between *Numba* and *MooCore* on ZCAT ($1.5\times$).

G. Cross-Framework Comparison

In this section, we first compare the runtime of VAMOS against other Python frameworks using generational NSGA-II. Then, we extend the comparison to NSGA-II variants (steady-state and external archive), SMS-EMOA, and MOEA/D.

Table IV compares the runtime of generational NSGA-II across five pure-Python frameworks and *PyGMO*, a compiled C++ baseline (Python bindings to the *pagmo2* library). Among pure Python frameworks, VAMOS with *Numba* backend is the fastest in all four benchmarks (ZDT, DTLZ, WFG, and ZCAT). *jMetalPy* is the closest competitor on ZDT, DTLZ, and ZCAT, yet it remains $2.2\text{--}4.9\times$ slower on those families and $29.4\times$ slower on WFG. *pymoo* is $2.7\text{--}8.2\times$ slower, while *DEAP* and *Platypus* are $4.3\text{--}82.9\times$ slower. *PyGMO* remains faster than VAMOS on ZDT, DTLZ, and WFG ($1.3\text{--}1.4\times$), as expected

given that both its algorithms and benchmark functions execute entirely in compiled C++ code; however, VAMOS is slightly faster on ZCAT (87.43 s vs. 98.55 s).

TABLE IV
NSGA-II WITH STANDARD SETTINGS. MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY ACROSS ALL FRAMEWORKS; PYGMO (C++) IS INCLUDED AS A COMPILED BASELINE

Framework	ZDT	DTLZ	WFG	ZCAT
VAMOS	4.89	8.14	14.33	87.43
<i>pymoo</i>	39.90	60.43	77.12	231.29
<i>jMetalPy</i>	24.09	32.30	421.20	196.02
<i>DEAP</i>	163.78	226.99	1154.80	378.59
<i>Platypus</i>	203.58	270.24	1188.44	445.80
<i>PyGMO</i> (C++)	3.41	5.69	11.07	98.55

We now compare the runtime of NSGA-II variants (steady-state and external unbounded archive), SMS-EMOA, and MOEA/D across the three frameworks that support them. Median values of these runtimes are reported in Table V. In the *steady-state* variant of NSGA-II, only one offspring is produced per iteration, so the main loop executes N times more iterations than the generational variant for the same evaluation budget, introducing additional per-iteration overhead. The variant with an *external unbounded archive* implies that whenever a new solution is evaluated, it is compared with the archive: if the new solution is non-dominated, it is inserted and any archive members it dominates are removed. This process becomes more time-consuming as the archive grows. Once the algorithm terminates, a subset of N diverse solutions is selected from the archive and returned as the final approximation set.

SMS-EMOA uses the hypervolume contribution as density estimator to select the worst individual to remove from the population in the replacement step, which is a time-consuming procedure given that the computation of the hypervolume contribution grows exponentially with the number of objectives. MOEA/D uses a decomposition strategy that, a priori, should not introduce any noticeable overhead.

TABLE V
MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY FOR NSGA-II VARIANTS, SMS-EMOA, AND MOEA/D.

Algorithm	Framework	ZDT	DTLZ	WFG	ZCAT
NSGA-II (steady-state)	VAMOS	150.21	244.80	799.29	715.29
	<i>pymoo</i>	5747.47	7259.88	6382.73	4569.01
	<i>jMetalPy</i>	1930.43	2506.83	2756.07	1947.76
NSGA-II (ext. archive)	VAMOS	10.43	87.56	23.33	106.30
	<i>pymoo</i>	1422.71	7208.12	1515.89	1511.36
	<i>jMetalPy</i>	744.93	3993.47	1736.74	602.88
SMS-EMOA	VAMOS	124.94	187.16	734.42	521.22
	<i>pymoo</i>	1658.10	2439.60	3410.49	2967.14
	<i>jMetalPy</i>	377.01	529.49	1161.51	2670.22
MOEA/D	VAMOS	76.19	118.98	647.96	340.78
	<i>pymoo</i>	1325.29	1735.46	4143.63	1023.63
	<i>jMetalPy</i>	303.00	430.14	4267.52	354.29

As shown in the above table, VAMOS is the fastest framework for all four algorithm variants across all problem

families. The speedup is most pronounced for the steady-state variant, where VAMOS is $6.4\text{--}38.3\times$ faster than *pymoo* and $2.7\text{--}12.9\times$ faster than *jMetalPy*. These large margins are expected: steady-state algorithms execute N iterations per generation, amplifying the per-iteration overhead of object-oriented frameworks. For the external-archive variant, VAMOS is $14.2\text{--}136.4\times$ faster than *pymoo* and $5.7\text{--}74.4\times$ faster than *jMetalPy*. For MOEA/D, VAMOS is $3.0\text{--}17.4\times$ faster than *pymoo* and $1.0\text{--}6.6\times$ faster than *jMetalPy*, with the narrowest gap on ZCAT. For SMS-EMOA, VAMOS is $4.6\text{--}13.3\times$ faster than *pymoo* and $1.6\text{--}5.1\times$ faster than *jMetalPy*, with the smallest margin on WFG.

Peak resident-set-size (RSS) measurements for NSGA-II on representative problems are provided in the Supplementary Material. These results consistently show that the array-native population model of VAMOS yields lower memory consumption than the object-centric designs used in other frameworks.

H. Convergence, Solution Quality, and Statistical Analysis

To verify that the runtime speedups do not compromise solution quality, the Supplementary Material provides three complementary analyses. The convergence analysis tracks normalized HV at 1,000-evaluation intervals over 50,000 evaluations on three representative problems (DTLZ2, WFG4, ZDT1). This analysis shows that VAMOS, *pymoo*, and *jMetalPy* follow nearly indistinguishable trajectories. The solution quality analysis reports that all frameworks attain virtually identical normalized HV on ZDT, medians are tightly clustered on DTLZ, and VAMOS achieves the highest family-level median on WFG for the NSGA-II baseline (0.934 vs. 0.846 for *pymoo* in the supplementary summary). The statistical analysis evinces that, for the 41 NSGA-II benchmark problems, Wilcoxon signed-rank tests with Holm correction reveal only one significant difference (ZDT6, $\Delta \approx 0.4\%$), and paired-bootstrap equivalence tests confirm that VAMOS is equivalent to *pymoo* on 37/41 problems and non-inferior on 38/41. Taken together, these results confirm that VAMOS delivers faster runtimes without sacrificing convergence behavior or final solution quality.

I. Discussion

The observed speedups are primarily due to (i) the array-based representation that removes per-solution Python overhead, and (ii) JIT compilation of hot operators and utilities in the *Numba* backend. The backend comparison in Table III is consistent with this interpretation, as the pure *NumPy* backend is substantially slower than *Numba* across all benchmark families. The solution quality analysis in the supplementary document summarizes that these performance gains are not obtained at the expense of final solution quality under the fixed-reference-front normalized-HV protocol, avoiding artifacts caused by run-dependent reference points.

The cross-framework gaps are problem-family dependent: WFG problems exhibit the largest relative speed decays for object-centric libraries, consistent with their higher dimensionality amplifying per-solution overheads. The magnitude of the speedup also varies across algorithm variants in a pattern

consistent with this overhead model. Steady-state variants (Table V) show the largest relative speedups because they perform one selection–variation–survival cycle per offspring; frameworks with high per-iteration Python overhead incur that cost 50,000 times (one per evaluation) rather than 500 times (one per generation of 100 offspring). The array-kernel dispatch featured by VAMOS amortizes this overhead effectively, widening the gap.

Not all comparisons yield equally large margins. The smallest gap appears for SMS-EMOA on WFG, where *jMetalPy* is only $1.6\times$ slower than VAMOS, and for MOEA/D on ZCAT, where the gap narrows to near parity ($1.04\times$). Unlike steady-state NSGA-II—where the *Numba*-accelerated ranking and crowding-distance computation of VAMOS dominate the per-iteration cost—the survival selection of SMS-EMOA is bottlenecked by the HV contribution calculation on the worst-ranked front. Both frameworks delegate this computation to the same C library (*moocore*), so the kernel acceleration of VAMOS yields a smaller differential advantage in that setting. Similarly, MOEA/D spends most of its computation latency in the neighborhood replacement stage rather than in generic ranking kernels, which limits the benefits of backend distinction on the ZCAT benchmark.

These multi-fold runtime reductions have direct practical implications. They mean that the same compute budget can accommodate more independent seeds, larger population sizes, or denser hyperparameter sweeps, precisely the requirements for statistically rigorous benchmarking studies.

J. Threats to Validity

Despite the careful alignment of dimensions, budgets, problem definitions, and operator settings, cross-framework comparisons remain subject to several threats: (i) operator implementations may differ in subtle details (e.g., boundary handling, duplicate elimination, tie-breaking in survival), (ii) random number generation and seeding behavior vary across libraries, and (iii) library-specific overheads (data conversion, per-solution object creation) may affect measured runtime. We mitigate major sources of unfairness by enforcing consistent benchmark bounds and matching operator behavior where APIs differ (e.g., mutation probability in *pymoo*).

Our cross-framework comparison focuses on Python MOEA libraries whose inner loops are predominantly executed in Python. *PyGMO* (Python bindings to the C++ *PaGMO2* library) is included as a compiled baseline for reference (Table IV), but it executes algorithms and benchmark functions fully in compiled code, yielding fundamentally different runtime profiles; its results are therefore not directly comparable with those of the pure-Python frameworks.

For hypervolume, reference fronts for problems without closed-form Pareto fronts are necessarily approximations; while we generate them densely and fix them across runs, remaining discrepancies can affect absolute normalized values, especially on WFG problems. We therefore treat normalized HV primarily as a comparative metric under a fixed protocol, and we report distributions across multiple seeds rather than relying on single-run outcomes.

Despite these limitations, the consistent trends across benchmark families and algorithm variants support the main conclusion that VAMOS substantially reduces runtime under the aligned protocol considered in this study.

VI. CONFIGURABILITY

We now examine two complementary aspects of the configurability of VAMOS. Section VI-A analyzes the parameter space of NSGA-II across frameworks, whereas Section VI-B describes how VAMOS integrates automated algorithm configuration tools to systematically explore that space.

A. Parameter Space Analysis of NSGA-II

We begin by analyzing the parameter space of NSGA-II with real-valued encoding. Table VI lists the 35 configurable parameters organized by component (general settings, initialization, selection, crossover, mutation, and repair). Of these, 11 are always active and 24 are conditional, i.e., they are activated only when their parent operator is selected. Component-wise, the 35 parameters comprise 5 general, 2 initialization, 2 selection, 15 crossover, 10 mutation, and 1 repair parameter. In any single configuration, between 11 and 19 parameters are simultaneously active, depending on the selected operators.

Table VII compares this breadth across frameworks (detailed per-framework tables are provided in the supplementary material). VAMOS offers 41 strategies and operators across the six components, roughly $3\times$ the variety of *pymoo* (14) and $1.6\times$ that of *jMetalPy* (26), with 24 conditional parameters that capture operator-specific settings. The largest gap appears in crossover (10 operators vs. 3 in *pymoo*) and mutation (9 vs. 2). Furthermore, no other framework exposes all configuration options as first-class parameters through a unified API: *pymoo* requires a custom `callback` for external archives, while *jMetalPy* needs an evaluator wrapper. VAMOS exposes all of them as builder-level options, enabling the systematic configuration-space exploration described in the next section.

The Supplementary Material contains the full parameter space of all the MOEAs included in VAMOS and an explanation of the parameters of each algorithm.

B. Automated Algorithm Configuration

The parameter spaces described in the previous section highlight a practical challenge: with 35 configurable parameters (spanning 41 strategies and operators) in the real-valued NSGA-II space, manually finding a good configuration for a given problem or problem family is impractical. Even experienced practitioners typically rely on well-known defaults (e.g., SBX with crossover distribution index $\eta_c = 20$, polynomial mutation with $p_m = 1/n$), which may be far from optimal for specific problem characteristics. Automated algorithm configuration addresses this challenge by systematically exploring the configuration space through efficient search procedures, thereby identifying high-performing settings without manual trial and error.

TABLE VI

PARAMETER SPACE OF NSGA-II WITH REAL-VALUED ENCODING (35 CONFIGURABLE PARAMETERS). PARAMETERS IN *ITALICS* ARE CONDITIONAL, AND VALUES IN **BOLD** DENOTE DEFAULTS. THE STEADY-STATE SETTING CORRESPONDS TO PRODUCING ONE OFFSPRING PER ITERATION. ACRONYMS IN THE TABLE DENOTE LATIN HYPERCUBE SAMPLING (LHS), OPPOSITION-BASED LEARNING (OBL), STOCHASTIC UNIVERSAL SAMPLING (SUS), PARENT-CENTRIC CROSSOVER (PCX), AND UNIMODAL NORMAL DISTRIBUTION CROSSOVER (UNDX).

Component	Value / Strategy	Conditional Parameters
General	Pop. Size $\in [20, 200]_{\mathbb{Z}}$ (log-scale); default 100	–
	Offspring Update $\in \{\text{steady-state}, 0.25, 0.5, 0.75, \mathbf{1.0}\}$	–
	Use External Archive $\in \{\text{True}, \mathbf{\text{False}}\}$	–
	Archive Mode $\in \{\mathbf{\text{bounded}}, \text{unbounded}\}$	<i>Archive</i> = True
	Archive Prune Policy $\in \{\mathbf{\text{crowding}}, \text{hv}, \text{mc_hv}, \text{knn}, \text{maxmin}, \text{ref_dirs}\}$	<i>Archive</i> = True, <i>Mode</i> = bounded
Initialization	{ Random , LHS, Scatter, Sobol, Halton, OBL}	–
	Scatter Base Size Factor $\in \{0.1, 0.2, 0.3, 0.5, 0.75, 1.0\}$	<i>Init.</i> = Scatter
Selection	{ Tournament , Random, Boltzmann, Ranking, SUS}	–
	Tournament Size $\in [2, 10]_{\mathbb{Z}}$	<i>Selection</i> = Tournament
Crossover	<i>Global</i> : Probability $\in [0.6, 1.0]_{\mathbb{R}}$	<i>Dist. Index</i> $\in [5.0, 40.0]_{\mathbb{R}}$
	SBX	$\alpha \in [0.0, 1.0]_{\mathbb{R}}$
	BLX- α	$\alpha \in [0.0, 1.0]_{\mathbb{R}}, \beta \in [0.0, 1.0]_{\mathbb{R}}$
	BLX- $\alpha\beta$	–
	Arithmetic	–
	Whole Arithmetic	$\alpha \in [0.0, 1.0]_{\mathbb{R}}$
	Laplace	$a \in [-1.0, 1.0]_{\mathbb{R}}, b \in [0.01, 2.0]_{\mathbb{R}}$
	Fuzzy	$d \in [0.0, 2.0]_{\mathbb{R}}$
	PCX	$\sigma_{\eta} \in [0.01, 0.5]_{\mathbb{R}}, \sigma_{\zeta} \in [0.01, 0.5]_{\mathbb{R}}$
	UNDX	$\zeta \in [0.1, 1.0]_{\mathbb{R}}, \eta \in [0.1, 1.0]_{\mathbb{R}}$
	Simplex	$\epsilon \in [0.1, 1.0]_{\mathbb{R}}$
Mutation	<i>Global</i> : Prob. Factor $\in [0.25, 3.0]_{\mathbb{R}}$ ($p_m = \text{factor}/n$)	–
	<i>Global</i> : Dist. Index $\in [5.0, 40.0]_{\mathbb{R}}$	–
	Polynomial	–
	Linked Polynomial	–
	Non-Uniform	<i>Perturbation</i> $\in [0.05, 0.5]_{\mathbb{R}}$
	Gaussian	$\sigma \in [0.001, 0.5]_{\mathbb{R}}$
	Cauchy	$\gamma \in [0.001, 0.5]_{\mathbb{R}}$
	Uniform	<i>Perturbation</i> $\in [0.01, 0.5]_{\mathbb{R}}$
	Uniform Reset	–
	Lévy Flight	$\beta \in [0.5, 2.0]_{\mathbb{R}}, \text{Scale} \in [0.001, 0.1]_{\mathbb{R}}$
	Power Law	<i>Index</i> $\in [0.5, 5.0]_{\mathbb{R}}$
Repair	{ clip , reflect, random, round, wrap, midpoint}	After crossover and mutation operations.

TABLE VII
NSGA-II CONFIGURATION BREADTH BY COMPONENT ACROSS
FRAMEWORKS (REAL-VALUED ENCODING)

Component	VAMOS	pymoo	jMetalPy
General	5 (2)	2	5 (2)
Initialization	6 (1)	2	2
Selection	5 (1)	2 (1)	3 (1)
Crossover	10 (13)	3 (6)	6 (9)
Mutation	9 (7)	2 (2)	6 (7)
Repair	6	3	4
Total	41 (24)	14 (9)	26 (19)

The optimization and machine learning communities have developed several well-established tools for hyperparameter search. Hyperopt [43] was among the earliest frameworks to introduce large-scale model search based on Tree-structured Parzen Estimators (TPE), demonstrating the effectiveness of Bayesian, non-parametric methods for navigating high-dimensional configuration spaces. Building on these ideas, SMAC3 [18] adopted Random Forest surrogate models together with an aggressive racing mechanism, becoming a widely used tool in the algorithm-configuration literature, particularly for settings with conditional and mixed-type pa-

rameters. More recently, Optuna [17] extended TPE-based optimization with a modern, lightweight architecture and additional sampling strategies, offering flexible distributed execution and strong performance in large search spaces. Complementing these Bayesian approaches, BOHB [19] integrates HyperBand’s multi-fidelity scheduling with Bayesian optimization, ensuring an efficient search by adaptively distributing compute budgets across competing configurations.

Rather than expecting users to install, configure, and interface with these tools independently, VAMOS integrates all three as tuning backends accessible through a unified API. The ModelBasedTuner facade provides a single entry point: users select the desired backend with a single parameter (`backend="optuna"`, `"smac3"`, or `"boh"`) and VAMOS handles the translation between its internal configuration space and the native format of each backend. This means that conditional parameter dependencies (such as the SBX distribution index being active only when SBX is the selected crossover operator) are automatically encoded in the search space of the backend without requiring any manual definition from the user. Optuna offers the broadest selection of sampling strategies (TPE, covariance matrix adaptation evolution strategy (CMA-ES), Gaussian Process, NSGA-II/III, Quasi-Monte

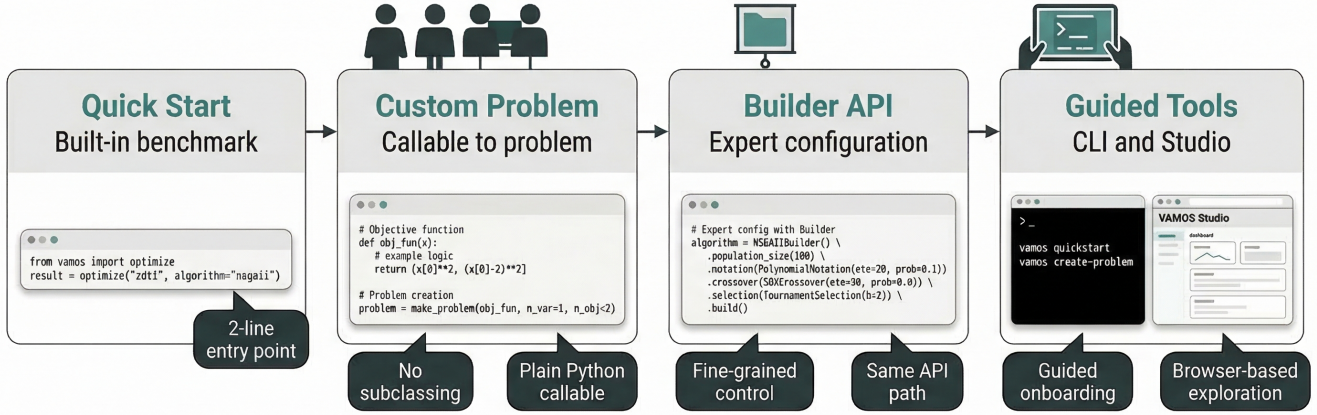


Fig. 4. Accessibility through progressive disclosure in VAMOS. The workflow spans a two-line quick start, callable-based custom problem definition without subclassing, fine-grained configuration through the builder API, and guided tooling through the command-line interface and VAMOS Studio.

Carlo) and supports distributed tuning via database-backed studies. SMAC3 excels in mixed configuration spaces where categorical, integer, and continuous parameters coexist with conditional dependencies. BOHB is particularly well suited when the evaluation budget varies, as its HyperBand scheduling automatically decides how many resources to allocate to each configuration.

VII. ACCESSIBILITY

A central design objective of VAMOS is to lower the entry barrier to multi-objective optimization while preserving the level of granular control required by expert practitioners. We interpret accessibility in terms of three common onboarding tasks that reflect typical first-use workflows: running a built-in benchmark, defining a small custom problem, and transitioning from default settings to explicit algorithm configuration. To assess these aspects, we curated canonical examples for VAMOS, *pymoo*, and *jMetalPy* based on the shortest documented pathways for each task, using these snippets as objective proxies for accessibility. Figure 4 summarizes the resulting progressive-disclosure workflow. Within the same unified framework, users can begin with a two-line quick start for built-in problems, proceed to callable-based problem definitions without subclassing, and ultimately access expert-level configuration through the builder API, supported by guided tooling via the command-line interface and VAMOS Studio.

TABLE VIII

ACCESSIBILITY PROXIES FROM CANONICAL ONBOARDING SNIPPETS ACROSS THE THREE CLOSEST PURE-PYTHON BASELINES

Framework	Built-in Run	Custom Problem	Expert Config	Guided Tools
VAMOS	2 (1)	11 (1)	16 (2)	✓
<i>pymoo</i>	6 (3)	12 (4)	13 (5)	✗
<i>jMetalPy</i>	6 (2)	22 (5)	17 (5)	✗

*: Blank lines, comments, plotting, and output-export code are excluded.

Table VIII summarizes the resulting proxy measurements. VAMOS has the lowest setup cost for the initial built-in run and for the callable-based custom-problem task, while *pymoo* becomes slightly shorter once the workflow shifts to

explicit operator-level configuration. We interpret this pattern as evidence for progressive disclosure rather than universal terseness: VAMOS minimizes the initial setup burden and then scales to expert control without leaving the public API, while also providing guided tooling beyond the Python interface.

```
1 from vamos import optimize
2
3 result = optimize("zdt1", algorithm="nsgaii")
```

Code 1. Minimal NSGA-II run on ZDT1 in VAMOS.

Code 1 shows the corresponding minimal VAMOS example. Executing a standard algorithm with sensible defaults on a built-in benchmark requires only two lines of code. For custom problems, `make_problem` converts a standard Python callable into a problem object without introducing framework-specific base classes, as shown in Code 2.

```
1 from vamos import make_problem, optimize
2
3 def my_objectives(x):
4     f1 = x[0]**2 + x[1]**2
5     f2 = (x[0] - 1)**2 + (x[1] - 1)**2
6     return [f1, f2]
7
8 problem = make_problem(
9     my_objectives, n_var=2, n_obj=2,
10    bounds=[(0, 2), (0, 2)],
11    encoding="real",
12 )
13 result = optimize(
14     problem, algorithm="nsgaii",
15     max_evaluations=50000, seed=42
16 )
```

Code 2. Defining and optimizing a custom MOP in VAMOS.

For advanced users who need systematic algorithm configuration and experimentation, VAMOS provides a unified builder API. Code 3 illustrates how a researcher can define the population size, configure specific crossover and mutation operators, and attach an external archive within the same execution model.

These examples illustrate the progressive-disclosure model of VAMOS: a built-in benchmark can be executed in two lines, a custom problem can be defined from a plain callable without subclassing, and the same API extends naturally to

```

1 from vamos import optimize
2 from vamos.algorithms import NSGAIConfig
3
4 cfg = (NSGAIConfig.builder()
5       .pop_size(100)
6       .crossover("sbx", prob=1.0, eta=20)
7       .mutation("pm", prob="1/n", eta=20)
8       .selection("tournament", size=4)
9       .external_archive()
10      .build())
11
12 result = optimize(
13     "dtlz1", algorithm="nsgaii",
14     algorithm_config=cfg,
15     max_evaluations=50000, seed=42
16 )

```

Code 3. Using the builder API for fine-grained configuration in VAMOS.

fully specified configurations when expert control is needed. Beyond the Python API, VAMOS also provides guided CLI entry points (e.g., `vamos quickstart` and `vamos create-problem`) and VAMOS Studio for browser-based onboarding and experiment exploration. The Supplementary Material details the canonical snippets and measurement rules used for the proxy comparison.

VIII. CONCLUSIONS

In this work we have presented VAMOS, a high-performance Python framework for multi-objective optimization studies. VAMOS addresses several long-standing limitations of existing Python-based MOEA libraries by combining a vectorized computational core, extensive configurability, and an accessible user experience within a single coherent design. Its array-native architecture, supported by pluggable numerical backends, substantially reduces runtime relative to alternative object-centric frameworks. The largest performance gains appear in steady-state and archive-intensive variants, while even the most computationally demanding cases (including SMS-EMOA on WFG or MOEA/D on ZCAT) show consistent improvements. Complementing this performance layer, VAMOS offers rich component-level configurability, exposing up to 36 unique parameters across supported encodings and 41 strategies and operators, roughly three times the variety available in contemporary Python frameworks. All components from initialization methods to variation operators and external archives are available as first-class configuration options through a unified builder API. Finally, VAMOS provides an accessible interface for users at different expertise levels, enabling complete optimization runs with only a few lines of code while maintaining a single path to advanced configuration. Automated configuration is also supported through native integration with Optuna, SMAC3, and BOHB, facilitating efficient exploration of conditional and high-dimensional MOEA parameter spaces.

Alongside the framework itself, we have designed a reproducible cross-framework benchmarking pipeline featuring a semantic-alignment protocol and fixed reference fronts for normalized HV computation. Using this protocol, our experiments have shown that VAMOS achieves solution quality levels that are statistically equivalent to state-of-the-art Python libraries

(equivalent on 37 of 41 NSGA-II problems and non-inferior on 38), while delivering substantial reductions in runtime. The convergence analysis and supplementary solution-quality results further demonstrate that these speedups do not compromise convergence behavior or final approximation quality. In conclusion, these results position VAMOS as a robust and versatile Python framework for future research, benchmarking, and methodological development in multi-objective optimization with MOEAs.

Looking ahead, several directions emerge for future work. Extending the current experimental protocol to many-objective optimization will require scalable performance indicators and may open new insights into the behavior of vectorized MOEAs under high dimensionality. Another promising avenue is the integration of distributed GPU execution, for example through coupling with frameworks such as EvoX [12], enabling multi-node acceleration for large-scale studies. Broader empirical comparisons, including additional frameworks, more MOEA variants, and complementary indicators, would further enrich the engine and experiment layers of the framework. Beyond performance and methodological rigor, enhancing user experience remains an important goal. We are investigating an LLM-assisted assistant capable of translating natural-language problem descriptions into validated experiment configurations, thereby lowering the barrier to entry for non-expert users. Formal user studies with domain practitioners from engineering, biology, and operations research would also provide valuable feedback on the usability of the API, the graphical interface, and the documentation. Finally, expanding the scope of the current benchmark to include binary, integer, and permutation encodings, as well as evaluating the declarative constraint-handling module on established constrained test suites, will help ensure that VAMOS provides robust support across the full breadth of multi-objective optimization scenarios.

DATA AND CODE AVAILABILITY

Code available upon acceptance.

REFERENCES

- [1] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester, UK: John Wiley & Sons, 2001.
- [2] C. A. C. Coello, G. B. Lamont, and D. A. V. Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2nd ed. New York, NY, USA: Springer, 2007.
- [3] S. Bleuler, M. Laumanns, L. Thiele, and E. Zitzler, "PISA—a platform and programming language independent interface for search algorithms," in *Proc. Evolutionary Multi-Criterion Optimization (EMO)*, ser. Lecture Notes in Computer Science, vol. 2632. Springer, 2003, pp. 494–508.
- [4] J. J. Durillo and A. J. Nebro, "jMetal: A Java framework for multi-objective optimization," *Advances in Engineering Software*, vol. 42, no. 10, pp. 760–771, 2011.
- [5] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "PlatEMO: A MATLAB platform for evolutionary multi-objective optimization," *IEEE Computational Intelligence Magazine*, vol. 12, no. 4, pp. 73–87, 2017.
- [6] J. Blank and K. Deb, "pymoo: Multi-objective optimization in Python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.
- [7] A. Benítez-Hidalgo, A. J. Nebro, J. García-Nieto, I. Oregi, and J. Del Ser, "jMetalPy: A Python framework for multi-objective optimization with metaheuristics," *Swarm and Evolutionary Computation*, vol. 51, p. 100598, 2019.
- [8] D. Hadka, "Platypus: A free and open source Python library for multiobjective optimization," GitHub repository, <https://github.com/Project-Platypus/Platypus>, 2015, accessed: 2026.

- [9] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.
- [10] H. Ishibuchi, L. M. Pang, and K. Shang, "A new framework of evolutionary multi-objective algorithms with an unbounded external archive," in *Proc. 24th European Conf. Artificial Intelligence (ECAI)*, 2020, pp. 283–290.
- [11] F. Biscani and D. Izzo, "A parallel global multiobjective framework for optimization: pagmo," *Journal of Open Source Software*, vol. 5, no. 53, p. 2338, 2020.
- [12] B. Huang, R. Cheng, Z. Li, Y. Jin, and K. C. Tan, "EvoX: A distributed GPU-accelerated framework for scalable evolutionary computation," *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 5, pp. 1649–1662, 2025.
- [13] Y. Tian, T. Zhang, J. Xiao, X. Zhang, and Y. Jin, "A coevolutionary framework for constrained multiobjective optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 1, pp. 102–116, Feb. 2021.
- [14] H. Zhao, X.-H. Ning, J.-Y. Li, and J. Liu, "Evolutionary multitask framework with bi-knowledge transfer for multimodal optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 30, no. 1, pp. 393–407, Feb. 2026.
- [15] H. Ye, H. Xu, and C. A. Coello Coello, "Light-EvoOPT: A lightweight evolutionary optimization framework for ultralarge-scale mixed integer linear programs," *IEEE Transactions on Evolutionary Computation*, vol. 30, no. 1, pp. 333–347, Feb. 2026.
- [16] Z. Ma, Y.-J. Gong, H. Guo, J. Chen, Y. Ma, Z. Cao, and J. Zhang, "LLaMoCo: Instruction tuning of large language models for optimization code generation," *IEEE Transactions on Evolutionary Computation*, 2026, early access.
- [17] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.
- [18] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, "SMAC3: A versatile Bayesian optimization package for hyperparameter optimization," *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022.
- [19] S. Falkner, A. Klein, and F. Hutter, "BOHB: Robust and efficient hyperparameter optimization at scale," in *International Conference on Machine Learning (ICML)*, vol. 80, 2018, pp. 1437–1446.
- [20] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [21] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *2nd Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, 2015.
- [22] M. Lopez-Ibañez *et al.*, "moocore: Multi-objective optimization core library," GitHub repository, <https://github.com/multi-objective/moocore>, 2024, accessed: 2026.
- [23] E. Zitzler, K. Deb, and L. Thiele, "Comparison of multiobjective evolutionary algorithms: Empirical results," *Evolutionary Computation*, vol. 8, no. 2, pp. 173–195, 2000.
- [24] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler, "Scalable multi-objective optimization test problems," in *IEEE Congress on Evolutionary Computation (CEC)*, 2002, pp. 825–830.
- [25] S. Huband, P. Hingston, L. Barone, and L. While, "A review of multiobjective test problems and a scalable test problem toolkit," *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 5, pp. 477–506, 2006.
- [26] S. Zapotecas-Martínez, C. A. Coello Coello, H. E. Aguirre, and K. Tanaka, "Challenging test problems for multi- and many-objective optimization," *Swarm and Evolutionary Computation*, vol. 81, p. 101350, 2023.
- [27] Q. Zhang, A. Zhou, S. Zhao, P. N. Suganthan, W. Liu, and S. Tiwari, "Multiobjective optimization test instances for the CEC 2009 special session and competition," University of Essex and Nanyang Technological University, Tech. Rep. CEC-MO09, 2008.
- [28] R. Tanabe and H. Ishibuchi, "An easy-to-use real-world multi-objective optimization problem suite," *Applied Soft Computing*, vol. 89, p. 106078, 2020.
- [29] S. Z. Martínez, A. García-Nájera, and A. Menchaca-Méndez, "Engineering applications of multi-objective evolutionary algorithms: A test suite of box-constrained real-world problems," *Engineering Applications of Artificial Intelligence*, vol. 122, p. 106192, 2023.
- [30] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [31] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.
- [32] E. Zitzler, M. Laumanns, and L. Thiele, "SPEA2: Improving the strength Pareto evolutionary algorithm," ETH Zurich, Zurich, Switzerland, Tech. Rep. TIK-Report 103, 2001.
- [33] A. Panichella, "An adaptive evolutionary algorithm based on non-Euclidean geometry for many-objective optimization," in *Genetic and Evolutionary Computation Conference (GECCO)*, 2019, pp. 595–603.
- [34] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.
- [35] R. Cheng, Y. Jin, M. Olhofer, and B. Sendhoff, "A reference vector guided evolutionary algorithm for many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 5, pp. 773–791, 2016.
- [36] N. Beume, B. Naujoks, and M. Emmerich, "SMS-EMOA: Multiobjective selection based on dominated hypervolume," *European Journal of Operational Research*, vol. 181, no. 3, pp. 1653–1669, 2007.
- [37] E. Zitzler and S. Künzli, "Indicator-based selection in multiobjective search," in *International Conference on Parallel Problem Solving from Nature (PPSN)*, 2004, pp. 832–842.
- [38] HoloViz, "Panel: The powerful data exploration & web app framework for Python," <https://panel.holoviz.org>, 2024, accessed: 2025-06-01.
- [39] X. Zhang, R. Cheng, Y. Tian, and Y. Jin, *Evolutionary Large-Scale Multi-Objective Optimization and Applications: Integrating Evolutionary Computation, Machine Learning, and Data Science*, 1st ed. John Wiley & Sons, 2024.
- [40] J. J. Durillo, A. J. Nebro, F. Luna, and E. Alba, "On the effect of the steady-state selection scheme in multi-objective genetic algorithms," in *International Conference on Evolutionary Multi-Criterion Optimization (EMO)*, 2009, pp. 183–197.
- [41] J. Blank, K. Deb, Y. D. Dhebar, S. Bandaru, and H. Seada, "Generating well-spaced points on a unit simplex for evolutionary many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 1, pp. 48–60, 2021.
- [42] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. Grunert da Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Transactions on Evolutionary Computation*, vol. 7, no. 2, pp. 117–132, 2003.
- [43] J. Bergstra, D. Yamins, and D. Cox, "Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures," in *International Conference on Machine Learning*, 2013, pp. 115–123.